

Karlsruhe Institute of Technology (KIT)

Modeling and Analysis in Mobility Software Engineering

Institute for Information Security and Reliability (KASTEL)

Jun.-Prof. Dr. Maike Schwammberger

76131 Karlsruhe

PRAXIS DER SOFTWAREENTWICKLUNG

UMLSL Traffic Editor

Winter Semester 2025/2026

Specifications Sheet

Supervisors: Qais Hamarneh, Dr. Akhila Bairy

Project Participants:

Name	Email Address
Florian Portscher	uymot@student.kit.edu
Joshua Kochhan	uinfy@student.kit.edu
Mattis Bihler	ufmuk@student.kit.edu
Tao Schmidt	uqgpk@student.kit.edu
Tim Kreuch	uvvuc@student.kit.edu

Karlsruhe, December 1, 2025

Table of Contents

1	Introduction and motivation	6
2	Determination of objectives	7
2.1	Mandatory criteria	7
2.1.1	Traffic Visual Editor	7
2.1.2	UMLSL Validator	7
2.1.3	Persistence	8
2.2	Desirable criteria	8
2.3	Optional criteria	8
2.4	Boundary criteria	9
3	Product Overview	10
3.1	Operating Conditions	10
3.2	Execution	10
3.3	Field of Application	10
3.4	Privileges and Security	10
3.5	Use Cases	11
3.6	Domain Data Models	14
3.6.1	Road attributes	14
3.6.2	Car attributes	14
4	User Interface Requirements	16
4.1	General Interface Structure	16
4.2	Global Control Requirements	18
4.3	Configuration Panel Requirements	19
4.3.1	Road Management	19
4.3.2	Car Management	19
4.3.3	Query Management (UMLSL)	19
4.4	Traffic Visual Editor Requirements	20
4.4.1	Visual Representation	20
4.4.2	Entity Selection	20
4.4.3	Updating View Position	20
4.5	Modal Editing Interfaces	20
4.5.1	Road Editor Overlay	20
4.5.2	Car Editor Overlay	20
4.5.3	UMLSL Query Overlay	21
4.6	Interaction Flow Requirements	21
5	Technical Specifications	22

5.1 Road Management	22
5.1.1 (UC1) Use Case: Add New Road	22
5.1.2 (UC2) Use Case: Edit Existing Road	22
5.1.3 (UC3) Use Case: Delete Existing Road	22
5.2 Car Management	24
5.2.1 (UC4) Use Case: Add New Car	24
5.2.2 (UC5) Use Case: Edit Existing Car	24
5.2.3 (UC6) Use Case: Delete Existing Car	24
5.3 UMLSL Management	26
5.3.1 (UC7) Use Case: Add New UMLSL Query	26
5.3.2 (UC8) Use Case: Edit Existing UMLSL Query	26
5.3.3 (UC9) Use Case: Delete Existing UMLSL Query	27
5.4 Element Selection	28
5.4.1 (UC9) Select Single Entity	28
5.4.2 (UC10) Deselect Single Entity	28
5.5 Viewport Transformation	29
5.5.1 (UC11) Zoom In/Out	29
5.5.2 (UC12) Move Vertical/Horizontal	29
5.6 System Settings	30
5.6.1 (UC13) Toggle Visualization of Car Safety Space	30
5.7 Persistence Functionality (F3)	31
5.7.1 (UC14) Save As Functionality	31
5.7.2 (UC15) Save Functionality	31
5.7.3 (UC16) Load From Functionality	31
6 Product Environment	33
6.1 Target Software	33
6.2 Hardware	33
6.3 Technology Stack (Development)	33
7 Non-functional Requirements	34
8 Global Test Scenarios	36
8.1 Functional Tests	36
8.1.1 Road Management	36
8.1.2 Car Management	36
8.1.3 UMLSL	37
8.1.4 Element Selection	37
8.1.5 Persistence	38

8.2 Non-functional Tests	38
9 Glossary	41

List of Figures

Figure 1 Main page of the application with the <i>Configuration Panel</i> on the left and the <i>Traffic Visual Editor</i> on the right.	6
Figure 2 Road Management	11
Figure 3 Car Management	11
Figure 4 Element Selection	12
Figure 5 Viewport Transformation	12
Figure 6 System Settings	13
Figure 7 Persistence	13
Figure 8 Sketch of all UI components	17

1 Introduction and motivation

In the field of Modeling and Analysis in Software Engineering, the formal verification of traffic scenarios is essential for ensuring the safety and reliability of autonomous systems. A key method for defining these spatial constraints is the Urban Multi-Lane Spatial Logic (UMLSL)¹. However, interpreting these logic queries against abstract text-based descriptions of traffic can be unintuitive and error-prone.

The UMLSL Traffic Editor aims to bridge the gap between formal logic and visual representation. The primary motivation of this project is to provide a standalone tool that allows users to model complex, two-dimensional traffic scenarios including multi-lane roads and vehicles and evaluate them using custom UMLSL query in real-time.

By integrating a *Traffic Visual Editor* with a query evaluator, the system enables users to verify whether specific traffic snapshots satisfy defined safety or operational conditions. This document outlines the software requirements specification of the UMLSL Traffic Editor software solution.

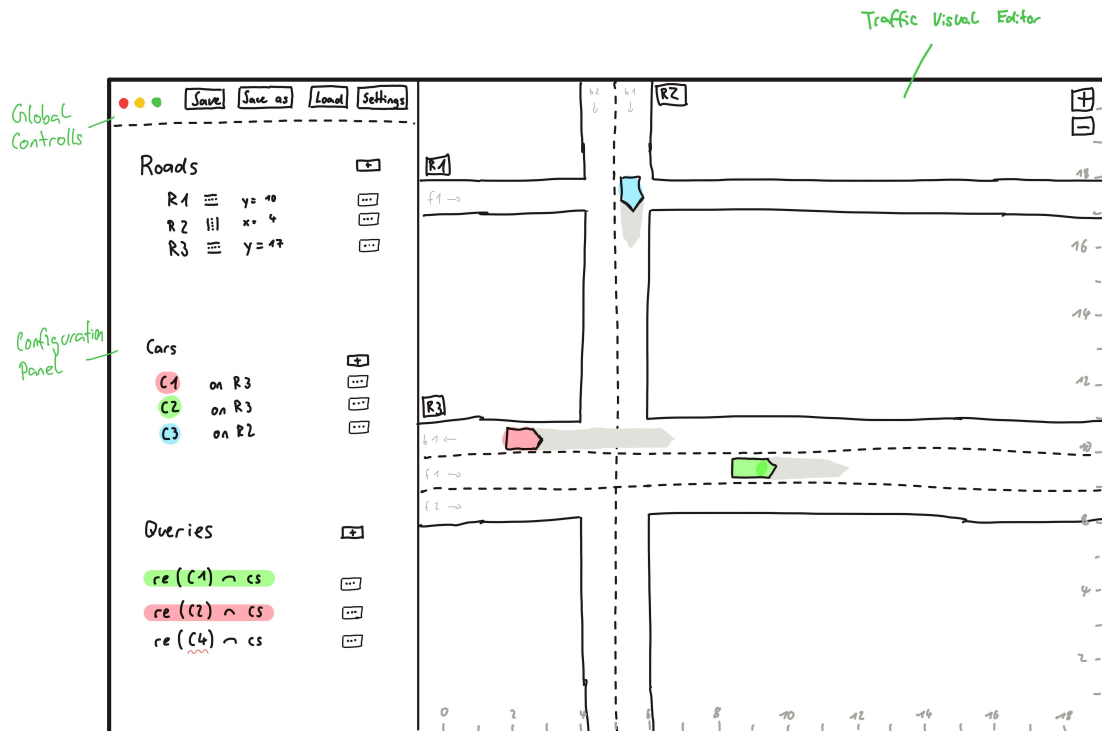


Figure 1: Main page of the application with the *Configuration Panel* on the left and the *Traffic Visual Editor* on the right.

¹Schwammberger, Maike. “An abstract model for proving safety of autonomous urban traffic.” Theoretical Computer Science 744 (2018): 143-169.

2 Determination of objectives

The application should provide a two-dimensional visual editor to model complex traffic scenarios and furthermore evaluate them with custom typed UMLSL queries. In the following paragraphs, the different high-level criteria and objectives are listed.

2.1 Mandatory criteria

2.1.1 Traffic Visual Editor

- (MC1) The program must support the placement of horizontal and vertical roads with a fixed number of lanes on a 2D canvas. Roads must be linear and infinite in length (explained more precisely in [Road Attributes](#)). All lanes have the same width and each lane has a direction (all lanes on one road are grouped by direction, ordered by right-hand traffic).
- (MC2) The user must be able to freely place cars on a lane.
- (MC3) The cars should have the following properties (explained more precisely in [Car Attributes](#)):
 - Name
 - Color
 - Road
 - Lane
 - Position on lane
 - Transition (value between -1 and 1 to describe lane changes)
 - Speed (can be negative to indicate opposite direction relative to the lane direction of the car)
 - Length (bounded length between 0.5 to 10.0 times the lane width)
 - Next Turn (Tuple of (*turning direction*, *next turn lane*), where the *turning direction* is the direction in which the car will turn, *next turn lane* being the lane after the car passed the intersection. *Next Turn* is *null* if there is no next turn.)
- (MC4) The intersection of two roads (i.e., a horizontally with a vertically oriented road) creates an intersection area (crossing).
- (MC5) Cars reserve their *safety space* (i.e., the space in front of the car equal to the car's braking distance, depending on the current speed and fixed braking acceleration (global constant)). The safety space is expanded over crossings if the car would otherwise stop on a crossing).
- (MC6) Cars can claim lanes next to them when starting a lane switch.
- (MC7) The user can configure whether *safety spaces* of all cars are visualized or not.

2.1.2 UMLSL Validator

- (MC8) The user can type any legal UMLSL query that refers to the state of the *Traffic Visual Editor* in the context of a car's view.

- (MC9) The application can evaluate all legal queries as true or false depending on the state of the *Traffic Visual Editor* after the user requests it. For illegal queries, an informative message is displayed.

2.1.3 Persistence

- (MC10) The application can save (save as) and load a *Traffic Snapshot* (i.e, a configuration that includes defined cars, roads, queries and the braking acceleration) to a file in persistent storage.
- (MC11) The configuration loader must provide error-correction capabilities for non-critical formatting or type errors, such as type conversion (e.g., auto-correct integer to double if double is expected).

2.2 Desirable criteria

- (DC1) User-friendly way to type in UMLSL queries by enabling the user to write custom LaTeX with the addition of a vertical and horizontal chop operation. Furthermore the LaTeX input gets rendered in a live preview.
- (DC2) After the initial placement and setting of parameters of cars and roads, the user has the possibility to update them later and change attributes.
- (DC3) Have multiple UMLSL queries active at the same time.
- (DC4) Validate active UMLSL queries automatically with any change of the visual editor state.
- (DC5) Visual distinction of different cars.
- (DC6) Zoomable canvas to zoom in and out in the *Traffic Visual Editor*.
- (DC7) Pannable canvas to move horizontally and vertically through space in the *Traffic Visual Editor*.
- (DC8) Save *Traffic Snapshot* to last saved file with one button click (quality of life feature, to not always select the file, where the user can store it)
- (DC9) Drag and drop to move/place roads and cars.
- (DC10) Single select an entity to change properties or delete it.
- (DC11) The *Traffic Visual Editor* includes a grid with xy-axes in the borders to aid positioning the entities.

2.3 Optional criteria

- (OC1) The application can simulate the progression of time for the state of the editor according to the properties of the cars (e.g., a play button; when the user presses it, all cars start driving according to their speed and turn at intersections according to their next turn property).
- (OC2) System-wide undo functionality (every change the user does that would affect the *Traffic Snapshot* can be undone and redone with Ctrl+Z and Shift+Ctrl+Z; e.g., when a

user changes the position of a car and then presses Ctrl+Z, the car gets placed to its original position).

- **(OC3)** Auto-saving (whenever the *Traffic Snapshot* changes, it gets saved automatically after a short debounce time to prevent too many saves in a short timeframe).
- **(OC4)** The grid of the the *Traffic Visual Editor* (DC11) can be toggled in the *global settings*.

2.4 Boundary criteria

- **(BC1)** Cars can drive through each other; no form of collision detection.
- **(BC2)** No traffic lights or road signs or any traffic rules.
- **(BC3)** No explicit turning lanes (a lane on which you have to turn at the intersection; these lanes are always before/in intersections) or curvy roads.
- **(BC4)** No prohibition of scenarios that would make no sense in real life; the user has full control over the traffic scenario (e.g., cars can drive and turn into lanes with a wrong direction).
- **(BC5)** A car can't perform a lane change inside a crossing.

3 Product Overview

This Chapter gives a brief overview of the usability and environment of the product. It explains how and by whom the system can be used and gives an introduction to its use cases.

3.1 Operating Conditions

- The program can be run at any time by a single user on a personal computer which has access to the application.
- There is no network connection needed to run the program since it does not rely on any type of server or database.

3.2 Execution

- The system can be started by executing the application.
- The system can be stopped by clicking a close button.
- The system will run in a standard window application with a graphical user interface.

3.3 Field of Application

- Targeted towards users who are familiar with UMLSL.
- Used to check UMLSL queries for a given traffic snapshot.
- Independent system which can be used as a tool in an environment together with other traffic simulation tools.

3.4 Privileges and Security

- The user has all privileges and can use any function the system offers.
- There is no administrator or any form of user hierarchy.
- The program can be run by anyone who has the application; there is no authentication or key required.
- There is no account system since only the local user matters.
- The application needs permission to local persistent storage to offer full functionality.

3.5 Use Cases

This section shows the use cases of the system. They are organized in different use-case diagrams based on how they occur in the program. They will be explained in more detail later in the “Technical Specifications”.

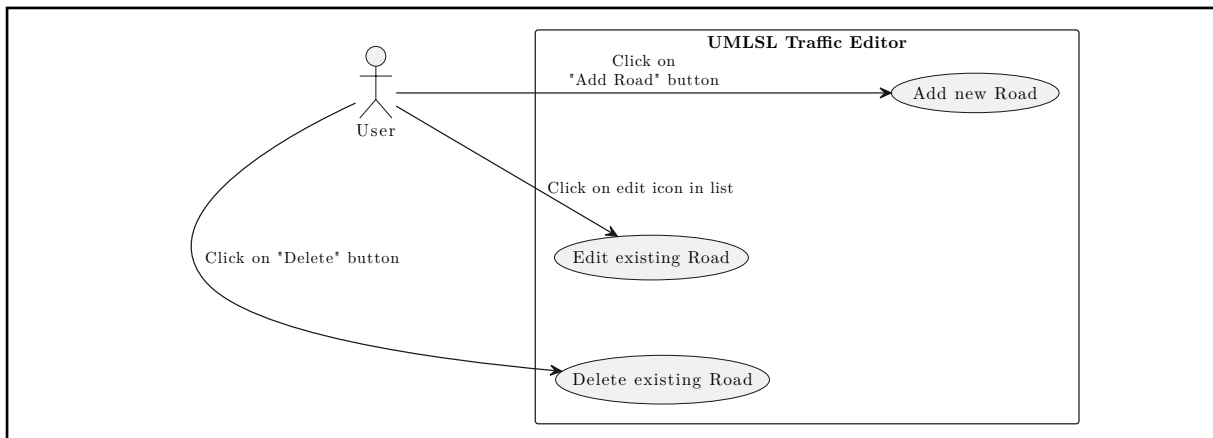


Figure 2: Road Management

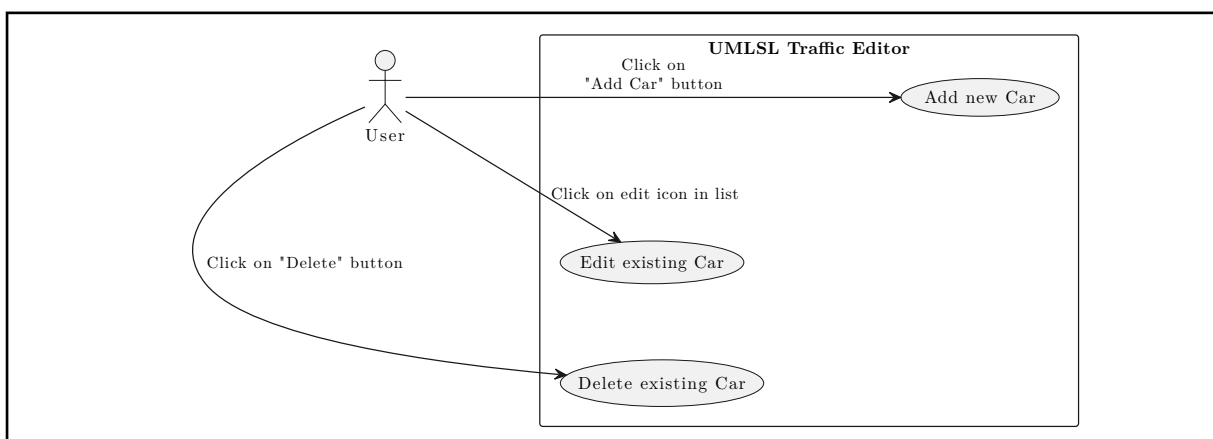


Figure 3: Car Management

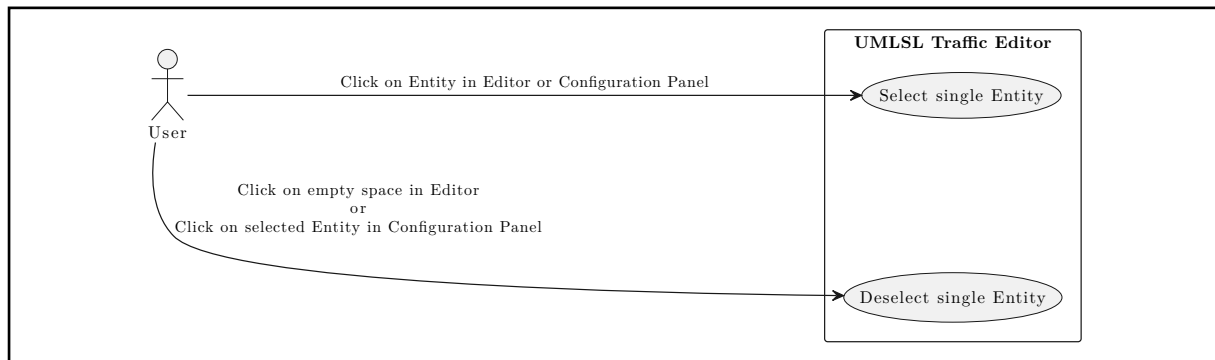


Figure 4: Element Selection

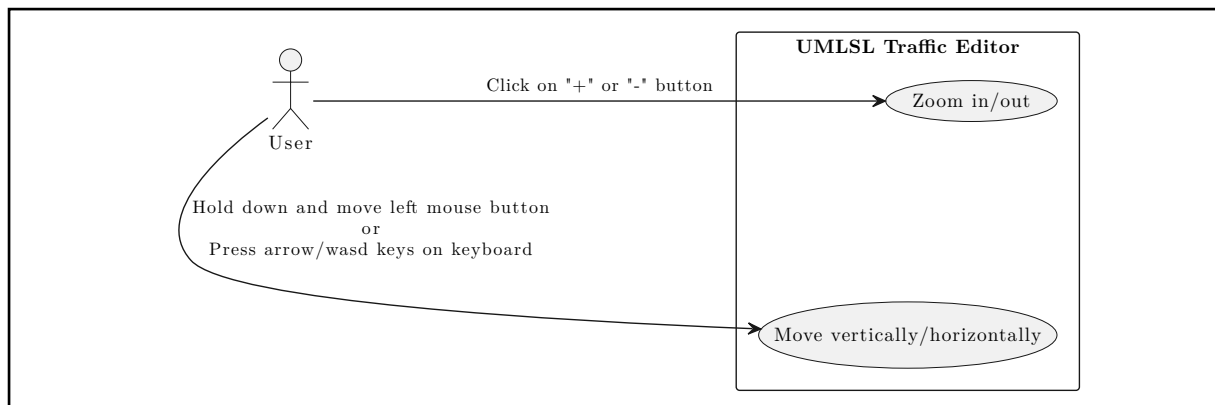


Figure 5: Viewport Transformation

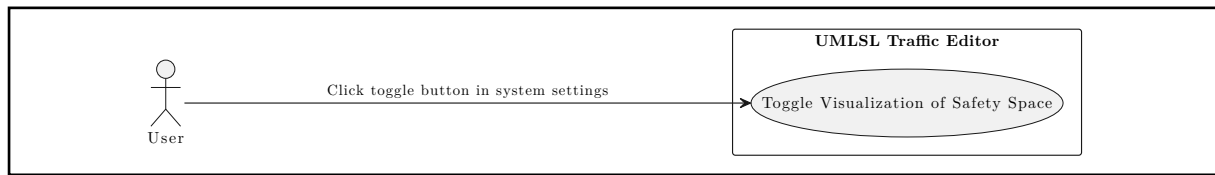


Figure 6: System Settings

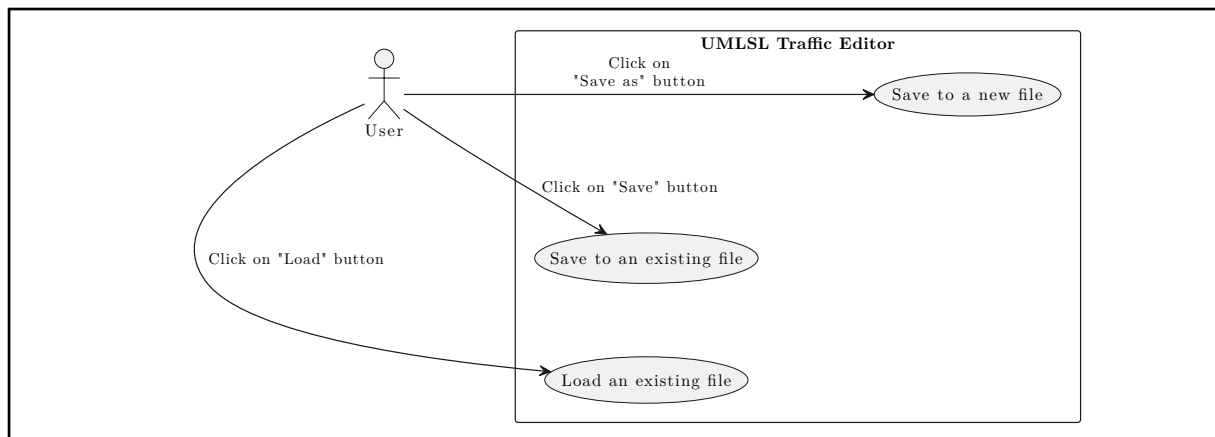


Figure 7: Persistence

3.6 Domain Data Models

In the following section, the attributes of the entities are further specified. All following sizes/distances/positions etc. are measured in units (u). One unit is exactly the width of one lane.

3.6.1 Road attributes

Name Describing the name of the road in a text format. The default name is “R{*road number*}” but it is changeable by the user.

Orientation Can be either *horizontal* or *vertical*.

Position The position is an integer (measured in units) that is either the x-position (in case of vertical orientation) or the y-position (in case of horizontal orientation). Only one coordinate is given, because every road has infinite length.

Number of Forward Lanes A number larger or equal to zero that describes the number of lanes in the right/up direction (whether the road is horizontal or vertical). The lanes are placed according to right-hand traffic (Germany/US), which means that all left lanes are placed on top of the right lanes, and all up lanes are placed on the right-hand side of all left lanes.

Every lane gets an automatic identifier: fn (forward) or bn (backward). Where n is the lane number starting at one, counting from the inside of the road. E.g. on a German Autobahn with 6 lanes, 3 right and 3 left, the lane for the trucks is $f3$ or $b3$ respectively, the middle lane $f2$ or $b2$ and the fastest lane is $f1$ or $b1$.

Number of Backward Lanes Same as “Number of Forward Lanes”, but in left/down direction.

The total number of lanes must be greater than zero.

3.6.2 Car attributes

Name Describing the name of the car in a text format. Gets automatically set with “C{*car number*}”, however, it is changeable by the user.

Color A text describing a hex code of the color in which the car should be visualized in the *Traffic Visual Editor*. The property is only for styling and has no effect on any application logic.

Road Must be one of the existing roads in the program, identifying the road by its name.

Lane The lane on which the car is driving; the lanes are identified by text: fn or bn .

Position On Lane The absolute x-position for horizontal roads and y-position for vertical roads. It is a floating-point number and can also be negative.

For example, if we have a horizontal road on $Position = 100$. The user places a car with *position on lane* = 50 in this road. The absolute position of the car is now $(50, 100 \pm \text{lane widths})$.

Transition A floating-point number between -1.0 and 1.0 (bounds not included) defaulting to 0.0 , which describes the amount of lane change a car has done, e.g. when the car is directly between both lanes, switching to the right, the transition value is 0.5 . The value must be between 0 and 1 (or -1 and 0 respectively) if the lane is on the outside of a road, so that the car cannot exit a road.

Speed A number which can also be negative, describing the speed of the car according to the lane direction. When the speed of the car is positive or zero, the car is positioned in the driving direction on the lane; when the speed is negative, the car is positioned against the direction of the lane.

Length A positive floating-point number in units which describes the length of the car, bounded between 0.5 and 10 . The width of a car is fixed to be slightly narrower than one lane.

Next Turn Tuple of (*turning direction*, *next turn lane*), null if the car makes no turn at the next intersection.

- *Turning direction*: The direction in which the car turns into the *next turn lane* (*turning direction* is either left or right).
- *Next turn lane* The lane of the next crossing road in which the car should turn with the given direction. Turning into a lane with the wrong direction is allowed. If the *Next Turn Direction* is straight, this attribute is not used because a car cannot perform a lane change inside a crossing.

4 User Interface Requirements

This chapter defines the functional requirements of the user interface. The UI enables users to add roads and cars, edit their attributes, and construct a traffic snapshot. It also allows users to evaluate queries on that traffic snapshot using the UMLSL language.

4.1 General Interface Structure

The application must provide a single coherent workspace divided into three primary functional areas:

Global Controls (Top Left) A section containing the save/load functions and general settings.

Configuration Panel (Left) A list-based editor for defining system entities (cars, roads, and queries).

Traffic Visual Editor (Right) A graphical representation of the *Traffic Snapshot*, where cars and roads are displayed.

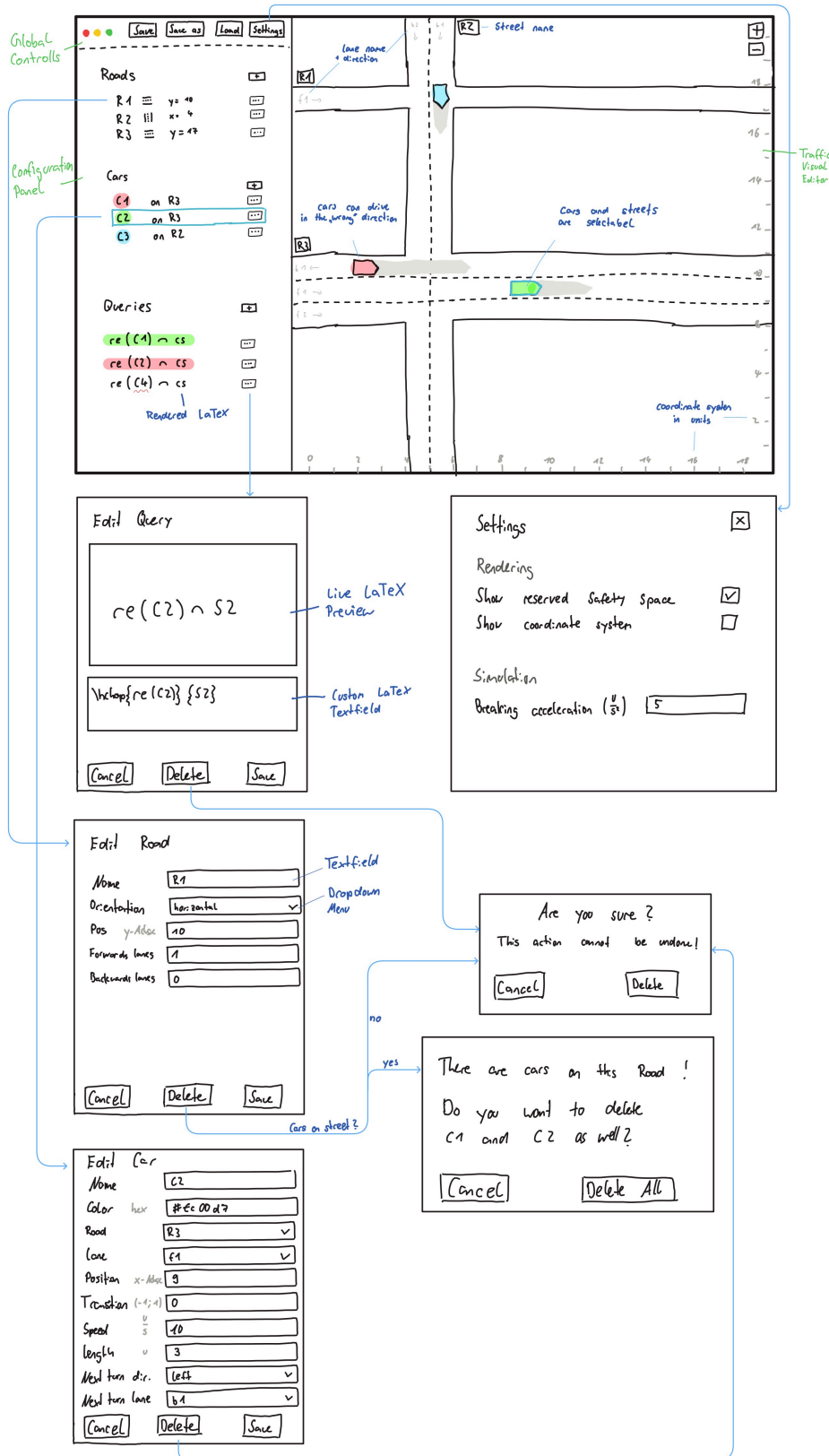


Figure 8: Sketch of all UI components

4.2 Global Control Requirements

The UI must provide access to application-level functions, permanently accessible or prominent within the workspace:

Persistence Operations Controls to save the state to the current file, a “save as” option to save to a new file, and load an existing configuration from a file. **(MC9)**

System Settings Access to global parameters affecting the simulation (e.g., braking acceleration) or visualization rendering, like displaying the safety space for cars.

4.3 Configuration Panel Requirements

The system shall provide a vertical sidebar dedicated to the management of simulation entities. This panel must be segmented into the following functional groups:

4.3.1 Road Management

List Representation The UI must list all defined road segments.

Identification Each road has its own name that matches the name tag in the *Traffic Visual Editor*.

Attribute Display Key topological parameters (e.g., Name, Orientation) must be visible at a glance.

Create, Edit, Delete Operations Users must be able to create new roads, edit existing attributes (e.g., lane count, coordinates), and delete empty roads. If the road isn't empty, the user is first asked if they want to delete all cars from that road. **(DC2)**

Selection Link Selecting a road in this list (clicking on the list tile) must visually highlight the corresponding element in the *Traffic Visual Editor*.

4.3.2 Car Management

List Representation The UI must list all vehicles currently in the simulation. **(MC2)**

Identification Each car has its own name and color that matches the representation in the *Traffic Visual Editor*. **(OC3)**

Attribute Display Key topological parameters (e.g., Name, current road) must be visible at a glance.

Create, Edit, Delete Operations Users must be able to create vehicles, modify their properties, and delete the entire car. **(MC2, DC2)**

Selection Link Selecting a car in this list (by clicking on the list element) must visually highlight the corresponding element in the *Traffic Visual Editor* area.

4.3.3 Query Management (UMLSL)

Query Input An editor interface must be provided to input queries using a custom LaTeX with horizontal and vertical chop operations. **(MC7, DC1, DC3)**

Evaluation Feedback The UI must display the Boolean result (True/False) of each query based on the current *Traffic Snapshot* (e.g., color-coding with green/red). **(MC8)**

Error Feedback If the query is invalid (e.g., it references a car that was removed after the query was defined), the UI should display an error indicator at the corresponding list item and an informative error message in the editor interface.

Create, Edit, Delete Operations User must be able to create new queries and edit existing ones.

4.4 Traffic Visual Editor Requirements

The right-hand section of the UI must render a 2D graphical interpretation of the abstract data defined in the Configuration Panel.

4.4.1 Visual Representation

Road Rendering Roads must be drawn according to their defined geometry (length, width, orientation) and intersections. Their name is drawn at the borders of the view. (MC1, MC4)

Lane Rendering Lanes are drawn with a width of one u and an arrow that points into the direction of the lane alongside its identifier at the borders of the view.

Car Rendering Cars must be displayed as distinct graphical objects with their colors at their calculated coordinates. (MC2, OC3)

Coordinate System If selected in the settings, the *Traffic Visual Editor* must show a grid with xy-axes in the borders to aid positioning the entities.

4.4.2 Entity Selection

Direct Selection Users must be able to select an entity directly within the *Traffic Visual Editor*.

Entity Highlighting The definition of the selected entity in the *Configuration Panel* is highlighted.

Entity Repositioning The user can change the position of the selected entity via drag and drop.

4.4.3 Updating View Position

The view must support zooming and panning to navigate larger topological networks. (OC2)

4.5 Modal Editing Interfaces

To accommodate complex property definitions without cluttering the main workspace, the application shall use modal overlays (popups) for specific create/edit actions.

4.5.1 Road Editor Overlay

Trigger Activated when clicking “Add” or “Edit” in the configuration panel.

Content The overlay must provide inputs for all required attributes. (MC1)

Validation The interface must prevent the entry of invalid topological data (e.g., negative lane counts).

4.5.2 Car Editor Overlay

Trigger Activated when clicking “Add” or “Edit” in the configuration panel.

Content The overlay must provide inputs for all required attributes. (MC2, MC3)

Precision The interface must allow for precise numerical input of simulation parameters. (DC2)

4.5.3 UMLSL Query Overlay

Trigger Activated when clicking “Add” or “Edit” in the configuration panel.

Input Area A text field allowing LaTeX-like free-text input of UMLSL syntax. **(MC7)**

Preview Area A distinct section that renders the LaTeX-style mathematical representation of the query in real-time as the user types. **(DC1)**

Action Controls Explicit “Save” and “Cancel” buttons to confirm or discard changes.

4.6 Interaction Flow Requirements

Synchronization Dragging or modifying an object in the *Traffic Visual Editor* must update the coordinate values in the *Configuration Panel* in real-time. **(OC4)**

Live Evaluation Any change to the traffic situation (whether via the panel or *Traffic Visual Editor*) must trigger an immediate re-evaluation of all defined UMLSL queries. **(DC4)**

5 Technical Specifications

5.1 Road Management

5.1.1 (UC1) Use Case: Add New Road

Use Case Add New Road

Objective Define a new road in *Traffic Visual Editor*.

Actors User

Triggering Event User clicks “Add Road” (+) in the Configurational Panel.

Postcondition (Success) A new road is drawn on the canvas and added to the list.

Main Flow

1. The system opens the Road Editor Overlay.
2. The user inputs all required attributes.
3. The user clicks “Save”.
4. The system draws the road and the road appears in the Configuration Panel.

5.1.2 (UC2) Use Case: Edit Existing Road

Use Case Edit Existing Road

Objective Change the attributes of an existing road.

Actors User

Precondition At least one road exists.

Triggering Event User clicks on the three dots in the list.

Postcondition (Success) The visualization redraws the road with its new attributes and all queries get reevaluated. If the number of lanes got decreased all cars on that deleted lanes get deleted.

Postcondition (Failure) If the input text is malformed or references non-existent lanes, an informative error message is displayed, and the road is visually marked as invalid (e.g., via red underline).

Main Flow

1. The system opens the Road Editor Overlay pre-filled with current data.
2. The user changes attributes.
3. The user gets potentially warned if cars on deleted lanes get deleted.
4. The user clicks “Save”.
5. The visualization redraws the road with its new attributes.

5.1.3 (UC3) Use Case: Delete Existing Road

Use Case Delete Existing Road

Objective Permanently remove a road from the simulation environment.

Actors User

Precondition At least one road exists.

Triggering Event User clicks on “Delete” in the Road Editor Overlay.

Postcondition (Success) The road is removed from the list and the visualization is updated.

Postcondition (Failure) If dependencies prevent deletion (e.g., cars are on the road), the user is first prompted with the option to delete all cars on that road.

Main Flow

1. The user clicks the delete button for a specific road.
2. The system checks for dependencies (cars assigned to this road).
3. If no dependencies exist, the system displays a confirmation dialog.
4. The user confirms the action.
5. The system removes the road.

Alternative Flow (Dependencies) 3a. If cars are on the road, the system displays a warning: “This road contains cars. Delete road and all associated cars?” 3b. The user confirms the deletion. 3c. The system deletes the associated cars and the road.

5.2 Car Management

5.2.1 (UC4) Use Case: Add New Car

Use Case Add New Car

Objective Place a new vehicle into the simulation with specific physical and kinematic properties.

Actors User

Precondition At least one road has been defined.

Triggering Event User clicks the “Add Car” (+) button in the Entity Management Panel.

Postcondition (Success) A new car appears in the visualization and is added to the sidebar.

Main Flow

1. The system opens the Car Editor Overlay.
2. The user inputs properties: Identifier, Velocity, Acceleration, ...
3. The user specifies the initial placement.
4. The user clicks “Create”.
5. The system renders the car in the Traffic Visual Editor and the car appears in the Configuration Panel.

5.2.2 (UC5) Use Case: Edit Existing Car

Use Case Edit Existing Car

Objective Modify the properties of a vehicle that is already in the simulation.

Actors User

Precondition At least one car exists in the simulation.

Triggering Event User clicks “Edit” in the list.

Postcondition (Success) The car’s physical behavior or position updates immediately and all UMLSL queries get reevaluated.

Main Flow (Dialog)

1. The system opens the Car Editor Overlay pre-filled with the car’s current data.
2. The user modifies a value (e.g., increases Velocity from 50 to 80).
3. The user clicks “Save”.
4. The visualization updates to reflect the change.

Alternative Flow (Direct Manipulation)

1. User selects a car in the Traffic Visual Editor.
2. User drags the car to a new location.
3. The system updates the coordinates in the Configuration Panel automatically.

5.2.3 (UC6) Use Case: Delete Existing Car

Use Case Delete Existing Car

Objective Permanently remove a vehicle from the simulation.

Actors User

Precondition At least one car exists.

Triggering Event User clicks “Delete” in the list.

Postcondition (Success) The car is removed from the visualization and list. All UMLSL queries are reevaluated.

Postcondition (Failure) If a UMLSL query explicitly references this car ID, the query is marked as invalid/error.

Main Flow

1. The user clicks the delete button for a specific car.
2. The system displays a confirmation dialog.
3. The user confirms the action.
4. The system removes the car and updates the visualization.

5.3 UMLSL Management

5.3.1 (UC7) Use Case: Add New UMLSL Query

Use Case Add New UMLSL Query

Objective Enable the user to enter a new UMLSL query using LaTeX-style syntax, which is automatically evaluated based on the current traffic snapshot.

Actors User

Precondition The application is running.

Triggering Event User clicks the “Add UMLSL” (+) button on the sidebar.

Postcondition (Success) The new UMLSL query is successfully validated, added to the list, and evaluated against the *Traffic Snapshot*.

Postcondition (Failure) If the input text is malformed or references non-existent cars, an informative error message is displayed, and the function is visually marked as invalid.

Main Flow

1. Upon triggering, a modal overlay opens containing an empty text input field and a LaTeX preview area.
2. The LaTeX renderer updates the preview in real-time as the user types into the text field.
3. The user clicks the “Add” button.
4. The system validates the syntax, stores the query, and evaluates it.

5.3.2 (UC8) Use Case: Edit Existing UMLSL Query

Use Case Edit Existing UMLSL Query

Objective Modify the syntax or logic of an already defined query.

Actors User

Precondition At least one UMLSL query exists in the Logic Definition list.

Triggering Event User clicks the “three dots” button next to a query in the Logic Definition Panel.

Postcondition (Success) The updated query is saved, replaced in the list, and immediately re-evaluated against the simulation state.

Postcondition (Failure) If the new syntax is invalid, the system retains the old query and displays an error in the overlay.

Main Flow

1. The system opens the UMLSL Query Overlay.
2. The text field is pre-filled with the existing query string.
3. The user modifies the text.
4. The LaTeX preview updates in real-time.
5. The user clicks “Save”.
6. The system validates and updates the query.

Alternative Flow (Cancel) 5a. The user clicks “Cancel”. 5b. The overlay closes and no changes are applied.

5.3.3 (UC9) Use Case: Delete Existing UMLSL Query

Use Case Delete Existing UMLSL Query

Objective Remove a logic query that is no longer needed.

Actors User

Precondition At least one UMLSL query exists.

Triggering Event User clicks the “Delete” icon next to a query in the Logic Definition Panel.

Postcondition (Success) The query is removed from the list.

Main Flow

1. The user clicks the delete button for a specific query.
2. The system removes the query from the list immediately.
3. The system stops evaluating that specific query.

5.4 Element Selection

5.4.1 (UC9) Select Single Entity

Use Case Select Single Entity

Requirements (DC10)

Objective Select a single entity (car or road) so that the user can edit its properties; the UI displays this selection.

Actors User

Precondition At least one entity exists and nothing is selected.

Triggering Event The user clicks on an entity in the Traffic Visual Editor or clicks on an entity in the Configuration Panel.

Postcondition (Success) The entity is selected and gets visually marked in the *Traffic Visual Editor* and the *Configuration Panel*.

Main Flow

1. The user clicks the entity.
2. The entity gets selected in the backend.
3. The selected entity gets marked visually.

5.4.2 (UC10) Deselect Single Entity

Use Case Deselect Single Entity

Objective Deselect a previously selected entity and remove the visual mark.

Actors User

Precondition The clicked entity is selected.

Triggering Event The user clicks the selected entity in the Traffic Visual Editor or in the Configuration Panel, or they click somewhere in empty space in the *Traffic Visual Editor*.

Postcondition (Success) The entity is no longer selected.

Main Flow

1. The user clicks the entity or the user clicks in empty space.
2. The entity gets deselected in the backend.
3. The previously visually marked entity is no longer marked.

5.5 Viewport Transformation

5.5.1 (UC11) Zoom In/Out

Use Case Zoom In/Out

Requirement (DC6)

Objective Scale up/down the *Traffic Visual Editor* viewport and all contents around the center of the screen portion which holds the editor, so that it appears to the user that they are zooming in or out.

Precondition None.

Postcondition Entire viewport is scaled up or down.

Actors User

Triggering Event User clicks the “+” or “-” button in the UI.

Main Flow

1. All elements in the viewport (cars/roads) get visually scaled up or down (attributes of entities stay fixed) by a fixed amount; optionally, this scaling is smoothed out (OC7).
2. Simultaneously translate the viewport when necessary so that the entities get scaled around the center of the visible area of the *Traffic Visual Editor* and not the center of the viewport.

5.5.2 (UC12) Move Vertical/Horizontal

Use Case Move/Pan Vertical/Horizontal

Requirement (DC7)

Objective Translate all entities in the *Traffic Visual Editor* in the same way, according to the input of the user, so that it appears to the user that they are moving through 2D space.

Precondition None.

Postcondition Entire viewport is translated horizontally or vertically.

Actors User

Triggering Event User pans in the viewport. Panning is defined as a gesture where the user holds the primary mouse button down and moves the pointer around the screen, or the user presses the arrow (or WASD) keys on the keyboard.

Main Flow

1. All elements in the viewport (cars/roads) get translated by the same amount according to the input, while clamping the bounds where the user can move so that they do not get lost on the infinite canvas.

5.6 System Settings

5.6.1 (UC13) Toggle Visualization of Car Safety Space

Use Case Toggle visualization of car safety space

Requirement (MC7)

Objective The user should decide whether the *Traffic Visual Editor* should or should not display the safety space of all vehicles.

Precondition None.

Postcondition The *Traffic Visual Editor* either displays the safety space or not.

Actors User

Triggering Event The user clicks the toggle in the System Settings and it gets automatically updated; no manual saving is needed.

5.7 Persistence Functionality (F3)

5.7.1 (UC14) Save As Functionality

Use Case Save the *Traffic Snapshot* to a new file

Objective Save the *Traffic Snapshot* of the current state as a new file in persistent data storage (e.g., the SSD of the computer where the application is running).

Precondition None.

Postcondition (Success) The *Traffic Snapshot* is successfully stored in persistent data storage under the given path and subtle feedback is provided to the user.

Postcondition (Failure) User gets notified that saving was unsuccessful.

Actors User

Triggering Event User clicks the Save As button in the UI.

Main Flow

1. Parse the *Traffic Snapshot* into a JSON format.
2. Communicate with the API of persistent data storage to store the data at a location the user picks.
3. After the saving attempt, return to the application and handle the outcome as described.

5.7.2 (UC15) Save Functionality

Use Case Save application state to last saved file

Objective Save the *Traffic Snapshot* of the current state into a file in persistent data storage (e.g. the SSD of the computer where the application is running).

Precondition A file has already been saved or loaded from persistent data storage during the runtime of the application. If not, execute (UC14) save as.

Postcondition (Success) *Traffic Snapshot* is successfully stored in persistent data storage and subtle feedback is provided to the user.

Postcondition (Failure) User gets notified that saving was unsuccessful.

Actors User

Triggering Event User clicks the Save button in the UI.

Main Flow

1. Parse the *Traffic Snapshot* into a JSON format
2. Communicate with the API of persistent data storage to store the data in the same path as the last saved file.
3. After the saving attempt, return to the application and handle the outcome as described.

5.7.3 (UC16) Load From Functionality

Use Case Load a traffic snapshot

Objective Load traffic snapshot from persistent data storage and apply it to the application.

Precondition Legal data is stored in persistent data storage.

Postcondition (Success) The loaded traffic snapshot is applied to the application and is fully usable, and the user gets subtle feedback that the loading was successful.

Postcondition (Failure) User gets notified that loading was unsuccessful.

Actors User

Triggering Event User clicks the Load button in the UI.

Main Flow

1. Ask the user if they want to save the current *Traffic Snapshot*, because otherwise it will be lost.
2. If the user declined saving or successfully saved their old state, the application communicates with the API of the persistent data storage to load the file the user selects in the process.
3. Loaded data is decoded and applied to the application while discarding the old data.

6 Product Environment

This chapter outlines the technical environment required for the operation and development of the UMLSL Traffic Editor. It defines the hardware and software prerequisites necessary for the end-user to execute the application, as well as the underlying technology stack used for implementation.

6.1 Target Software

The application is distributed as a runnable Python script (.py). Therefore, a Python runtime environment (Version 3.9 or newer) is required on the host machine to execute the software. The application is compatible with the following operating systems:

- Windows: Windows 10 or later.
- macOS: macOS X or later.
- Linux: Current standard distributions (e.g., Ubuntu, Fedora).

The application requires read/write access to the local file system to perform save and load operations.

6.2 Hardware

The application is designed to run on standard personal computer hardware. No dedicated high-performance components (e.g., dedicated GPU) are required.

- Peripherals: A keyboard and mouse (or trackpad) are mandatory for interaction.
- Display: A monitor is required to visualize the Graphical User Interface.

6.3 Technology Stack (Development)

The development and execution of the UMLSL Traffic Editor relies on the following technologies:

- Programming Language: Python 3.9.x² or newer. Used as the main programming language for the entire application. We selected it due to the developers' prior knowledge and the general ease of use of the programming language for an application with no focus on high performance.
- GUI Framework: PyQt6³ (requires Python 3.9 or higher). We selected it because it is the industry standard for desktop applications and for its ease of use.
- External Libraries: Matplotlib⁴ to render the live LaTeX preview. We selected it because it is the only well-known and supported publicly accessible Python package that supports LaTeX rendering.

²<https://www.python.org/>

³<https://pypi.org/project/PyQt6/>

⁴<https://matplotlib.org/>

7 Non-functional Requirements

ID	Name (Category)	Description	Acceptance
NFR-01	UI Response Time (Performance)	The UI shall provide an interactive experience for the user.	The UI gives visual feedback in a reasonable time after the user interaction.
NFR-02	Decoupling Architecture (Structure)	The application shall account for potential UI changes.	Frontend (UI) and Backend (Logic) are separated and access to the Backend must occur through an API.
NFR-03	Documentation (Maintainability)	The source code shall be documented to ensure long-term maintainability.	The source code contains documentation for at least all non-trivial functions (e.g., query evaluation).
NFR-04	Test Coverage (Reliability)	Automated tests shall be included to detect malfunctioning code.	UMLSL parser/validator has at least 70% automated test coverage.
NFR-05	Modal Creation (Reliability)	The application shall prevent an overload of modal creations.	No more than one modal can refer to the same entity.
NFR-06	Crash Robustness (Reliability)	Invalid inputs (cars, roads, queries, global settings, <i>Traffic Snapshots</i>) shall never crash the application.	The program must account for invalid inputs and must display an informative error to prevent application crashes.
NFR-07	Self-explanatory UI (Quality)	For users familiar with UMLSL, the UI shall be usable without reading its documentation.	Create a usability test: An arbitrary person familiar with UMLSL

ID	Name (Category)	Description	Acceptance
			must succeed in 80% of tasks.
NFR-08	Cross-Platform Compatibility (Portability)	Ensures wide usability.	The application runs on: • Windows10 and 11 • macOS X • Ubuntu and Fedora
NFR-09	Scenario Scalability (Scalability)	The application shall handle large traffic models.	Application remains operational with ≥ 200 cars and ≥ 50 roads with reasonable UI response times.
NFR-10	Storage Format (File Management)	The <i>Traffic Snapshot</i> shall be in a human-readable form.	Use JSON.

8 Global Test Scenarios

This chapter defines the test scenarios used to verify the functional and non-functional requirements of the UMLSL Traffic Editor. The tests are derived from the Use Cases defined in Section 4 and the Non-functional Requirements defined in Section 5.

8.1 Functional Tests

The following functional sequences verify the core interactions of the system.

8.1.1 Road Management

(T10) Test Case for UC1 (Add New Road): Action: The user clicks “Add Road”, enters valid attributes within legal bounds in the overlay, and clicks “Create”.

Expected Result: A new road appears on the canvas with the specified properties. A corresponding entry appears in the “Roads” list in the Configuration Panel.

(T20) Test Case for UC2 (Edit Existing Road):

Action: The user selects an existing road (S1), clicks “Edit”, changes some attributes within the legal range, and clicks “Save”.

Expected Result: The visualization of S1 updates immediately to display the new attributes. The attributes in the Configuration Panel reflect the new values.

(T30) Test Case for UC3 (Delete Existing Road):

Action: The user selects an existing road and clicks the “Delete” button.

Expected Result: After a secondary deletion verification by the user, the road and the cars on it disappear from the visualization area, and the items get removed from the road and car list in the Configuration Panel.

(T31) Error Handling Road Input:

Action: The user attempts to create/update a road with illegal attributes.

Expected Result: The system blocks the creation and displays an informative error message in an overlay.

8.1.2 Car Management

(T40) Test Case for UC4 (Add New Car):

Action: The user clicks “Add Car”, specifies attributes, sets a position, and clicks “Create”.

Expected Result: Car C1 is rendered on road R1 at the specified position. The car is listed in the “Cars” section of the Configuration Panel.

(T50) Test Case for UC5 (Edit Existing Car):

Action: The user double-clicks Car C1, changes some attributes within the legal range, and clicks “Save”.

Expected Result: The car's properties are updated. If a UMLSL query depends on this velocity, the query's evaluation result is updated immediately.

(T60) Test Case for UC6 (Delete Existing Car):

Action: The user selects Car C2 and clicks the "Delete" button.

Expected Result: Car C2 is removed from the canvas and the configuration list. Any UMLSL queries referencing C2 evaluate to an error.

(T61) Error Handling Car Input:

Action: The user attempts to create/update a car with illegal attributes.

Expected Result: The system blocks the creation and displays an informative error message in an overlay.

8.1.3 UMLSL

(T70) Test Case for UC7 (Add New UMLSL Query):

Action: The user clicks "Add UMLSL", types a valid query in the input field, and clicks "Add".

Expected Result: The query appears in the UMLSL list. The preview shows the rendered mathematical syntax. The evaluation gets evaluated and visually displayed based on the current state.

(T80) Test Case for UC8 (Edit Existing UMLSL Query):

Action: The user edits an existing query to introduce a syntax error (e.g., missing closing parenthesis) and attempts to save.

Expected Result: The system displays an error message in the overlay and prevents saving until the syntax is corrected.

(T85) Real-time Evaluation:

Action: A query that references Car C1 is currently true. The user edits Car C1 and changes some attributes.

Expected Result: Immediately upon changing the car's new attributes, the query gets evaluated again and the outcome gets displayed.

8.1.4 Element Selection

(T90) Visual Selection Sync:

Action: The user clicks on a car in the *Traffic Visual Editor*.

Expected Result: The corresponding entry in the Configuration Panel is highlighted.

Action: The user clicks on an entry in the road list.

Expected Result: The corresponding road on the canvas is visually highlighted.

(T95) Drag and Drop Position Update:

Action: The user drags a car on the canvas to a new position.

Expected Result: The position coordinates in the Configuration Panel update in real-time during the drag operation.

(T96) Error Handling Invalid Drag and Drop:

Action: The user drags a car on the canvas to a new position which is not on a road (illegal position).

Expected Result: The system displays an informative error message and the car gets placed back to its original position.

8.1.5 Persistence

(T100) Test Case for UC13 (Save As):

Action: The user clicks “Save As”, enters a filename, and confirms.

Expected Result: A file is created in the persistent storage containing the data of all roads, cars, and queries. The user receives feedback that the save was successful.

(T110) Test Case for UC14 (Save):

Action: After modifying a loaded scenario, the user clicks “Save”.

Expected Result: The changes are written to the currently active file without prompting for a name.

(T120) Test Case for UC15 (Load):

Action: The user clicks “Load” and selects a valid save file.

Expected Result: The current editor state is cleared after confirmation from the user and replaced entirely by the entities and queries defined in the file.

8.2 Non-functional Tests

(TNFR-01) Test Case for NFR-01: UI Response Time (Performance)

Description: Automated performance test measuring the latency between a user interaction (e.g., clicking “Select”) and the visual frame update.

Constraint: The frame update must be within a reasonable time after the user interaction.

(TNFR-02) Test Case for NFR-02: Decoupling Architecture (Structure)

Description: Perform static code analysis or architectural review.

Acceptance: Verify that the backend (logic) files do not use/import UI libraries and review that the frontend (UI) accesses the backend only through defined API interfaces.

(TNFR-03) Test Case for NFR-03: Documentation (Maintainability)

Description: Manually review files containing non-trivial code (e.g., query evaluator).

Acceptance: Every one of these functions must include thorough documentation.

(TNFR-04) Test Case for NFR-04: Test Coverage (Reliability)

Description: Run automated test coverage tools (e.g., `coverage.py`⁵ for Python).

Acceptance: UMLSL parser/validator achieves at least 70% line coverage.

(TNFR-05) Test Case for NFR-05: Modal Creation (Reliability)

Description: Open the “Edit Road” modal for road S1. Without closing it, attempt to trigger the “Edit Road” modal for S1 again (if UI allows) or check internal state managers.

Acceptance: The system ensures only one modal instance exists for that entity, preventing duplicate overlays or state conflicts.

(TNFR-06) Test Case for NFR-06: Crash Robustness (Reliability)

Description: Consider all input fields in the “Edit Car”, “Edit Road”, “Edit Query” modals as well as in the *Global Settings* (i.e., the braking distance field) and type in values that mismatch the expected type as specified in the requirements, reference non-existing entities, and type in invalid LaTeX code (specifically for “Edit Query”).

Acceptance: The program recognizes the invalid input and displays a meaningful error message clearly visible to the user.

(TNFR-07) Test Case for NFR-07: Self-explanatory UI (Quality)

Description: Conduct a simple usability test with a person familiar with UMLSL that has not seen the documentation of this application.

Acceptance: Create a basic task that includes:

- Creation of a traffic snapshot that contains at least one road and one car.
- Multiple queries for the traffic snapshot.
- Modifying the traffic snapshot (at least one car, one road, and one query).
- Storage of the existing traffic snapshot.
- Loading a given *Traffic Snapshot*.
- Toggling the option to show safety spaces in the *Global Settings*.

The user must succeed in 5 of the total 6 exercises.

(TNFR-08) Test Case for NFR-08: Cross-Platform Compatibility (Portability)

Description: Run the application on:

- Windows: Windows 10 and Windows 11.
- macOS: macOS X.
- Linux: Ubuntu and Fedora.

Acceptance: The application must run successfully on all platforms, especially without platform-specific UI issues (rendering or interaction flaws).

(TNFR-09) Test Case for NFR-09: Scenario Scalability (Scalability)

⁵<https://github.com/coveragepy/coveragepy>

Description: Manually create a large traffic snapshot with 200 cars and 50 roads and check the UI response time when defining additional entities to that snapshot.

Acceptance: The UI response time is reasonable (non user disturbing) when defining these additional entities in that snapshot.

(TNFR-10) Test Case for NFR-10: Storage Format (File Management)

Description: Save a scenario to disk and open the resulting file in a text editor.

Acceptance: The file content must be valid JSON syntax and human-readable (keys are descriptive strings, structure is hierarchical).

9 Glossary

Car A vehicle in the traffic simulation which has specific kinetic and topological properties.

Debounce Time The debounce time is a minimum duration (often measured in milliseconds) that a system must wait after the last user input or activity before it triggers a specific, resource-intensive action, such as auto-saving data.

Entity Throughout the entire document, “entity” is used as a superordinate term for roads and cars.

Global Settings Application-wide configuration parameters that affect simulation physics (braking acceleration) or visualization options (rendering of safety spaces and - if OC4 holds - rendering of the grid).

Lane A specific directional part of a road to which cars can be assigned.

Panning Panning allows you to slide your view across the screen horizontally or vertically, similar to moving a large sheet of paper across a desk to work on a specific spot. It helps you navigate to different parts of your canvas without changing the magnification level. Crucially, this moves your perspective only, not the actual objects in your design.

Persistence The system’s ability to save and load its current state in a file system for later reuse.

Road A straight, vertical or horizontal traffic element with fixed width and a specific amount of lanes in different directions. It contains the cars of the simulation.

Safety Space The safety space is a space reserved by cars in front of them. Under straight lane conditions, the distance of the safety space equals the braking distance. When encountering a crossing, the safety space is expanded along the desired lane of the car after the crossing. The visualization of safety spaces of all cars can be toggled in the Global Settings.

Traffic Snapshot A Traffic Snapshot contains all defined cars, roads, queries, that are currently existing in the application as well as the braking distance.

Traffic Visual Editor Refers to a window on the right-hand side of the application which renders the current Traffic Snapshot according to the settings specified by the user.

Traffic Visual Editor Coordinate System A 2D Cartesian coordinate system used within the Traffic Visual Editor to position roads and cars, measured in units (u).

UMLSL Urban Multi-Lane Spatial Logic. It is a semantic language used to evaluate traffic scenarios with straight roads and cars. Introduced in: Schwammberger, Maike. “An abstract model for proving safety of autonomous urban traffic.” Theoretical Computer Science 744 (2018): 143-169.

Zooming Zooming changes the magnification of your view, acting like a digital magnifying glass. You can “zoom in” to see fine details clearly or “zoom out” to view the entire project

at a glance. While this changes how large items appear on your screen, it does not alter their actual real-world dimensions or print size.